

数据同步服务

一.数据同步服务的基本介绍

- 1.什么是数据同步？
- 2.如何进行实时同步
- 3.特点
- 4.怎么实现仅同步变化的数据呢？
- 5.同步方式

三.综合案例

1. 本地同步操作:
- 2.远程同步操作:
- 3.综合案例,实现定时与增量备份

四.结合INOTIFY实现数据实时同步

- 1.对node2进行配置
 - 1.创建用户
 - 2.修改配置文件
 - 3.创建密码文件
 4. 启动rsync服务

五.设置开机自启动

- 1.杀死进程
- 2.设置开机自启动

六.基于RSYNC服务完成手动同步

- 1.格式
- 2.基于手动同步方式完成数据同步

3.创建日志文件

- 4.开始传输文件了

解决方案

找到根本原因了！

解决方案

方案1：临时关闭 SELinux（快速验证）

方案2：永久关闭 SELinux

方案3：设置正确的 SELinux 上下文（推荐，更安全）

建议

七.使用密码文件， 实现不需要输入密码

八.使用主机名

九.使用脚本自动配置

- 1.安装使用的软件
- 2.编写脚本

3.给文件加权以及启动脚本文件

4.最后测试一下

一.数据同步服务的基本介绍

1.什么是数据同步？

指的将业务中数据，定时 或者实时的将其备份到其他目录或者其他服务器的目录下过程

2.如何进行实时同步

常用选项：

- `-v` ：详细模式输出，显示传输过程中的文件信息。
- `-a` ：归档模式，表示递归传输文件，并保持所有文件属性。
- `-z` ：对备份的文件在传输时进行压缩处理。
- `-P` ：显示进度条。
- `--delete` ：删除目标目录中源目录中没有的文件（可选，用于实现双向同步）
- `-e` ：用于指定用于传输文件的远程 shell 程序 默认为 `ssh`，也可以选择其他如`rsh`或者自定义的`ssh`

端口

本地同步：

```
rsync -av /src/directory/ /dest/directory/
```

远程同步：

```
rsync -avz /src/directory/ root@192.168.88.102:/dest/directory/
```

拉取远程数据到本地：

```
rsync -avz user@192.168.88.102:/src/directory/ /dest/directory/
```

3.特点

- 支持增量同步：只传输发生变化的数据部分。
- 支持 SSH 加密传输。
- 可保持文件权限、时间戳、连接和等元数据。

4.怎么实现仅同步变化的数据呢?

1. 会对需要同步的数据文件进行分块(底层)
2. 每一个数据块会基于数据生成校验码，只要数据发生变化 这个校验码也会发生改变
3. 在同步文件的时候， 只需要监测哪些数据块的校验码发生变化， 就可以指定那个块数据改变了， 此时仅需要同步这些变化数据块即可

5.同步方式

- 手动同步--> 定时同步
- 实时同步

二.如何安装数据同步服务： `dnf -y install rsync`

```
dnf install rsync -y
```

```
[root@192 backup]# dnf install rsync -y
上次元数据过期检查：0:49:08 前，执行于 2026年07月24日 星期五 11时23分24秒。
软件包 rsync-3.2.5-9.el9.x86_64 已安装。
依赖关系解决。
无需任何处理。
完毕！
[root@192 backup]# █
```

三.综合案例

1. 本地同步操作:

准备工作

1- 在node1节点的 /usr/local目录下,创建一个nginx-1.8.0目录,并在此目录下构建一个html目录

2- 在html目录下,创建index.html、favicon.ico、logo.png文件,以及assets目录

3- 在assets目录下,创建index.js、index.css文件

4- 给各个文件随机写入一些内容

需求: 对html目录进行同步操作。 将其同步到/usr/local/nginx-1.8.0/backup/下, 并且如果源目录下已经删除的, 在目标目录下也应该被删除

我们应当用rsync命令进行相应的操作, 下面就是我操作出来的

```
[root@192 ~]# mkdir -p /usr/local/nginx-1.8.0
[root@192 ~]# cd /usr/local/nginx-1.8.0
[root@192 nginx-1.8.0]# cd ..
[root@192 local]# ls
bin  etc  games  include  lib  lib64  libexec  nginx-1.8.0  sbin  share  src
[root@192 local]# cd nginx-1.8.0/
[root@192 nginx-1.8.0]# ls
backup  html
[root@192 nginx-1.8.0]# mkdir html
mkdir: 无法创建目录“html”: 文件已存在
[root@192 nginx-1.8.0]# cd html
[root@192 html]# touch index.html favicon.ico logo.png
[root@192 html]# mkdir assets
```

```
[root@192 assets]# echo "
bin  etc  games  include  lib  lib64  libexec  n" | tee -a index.js index.css ../index.html ../favicon.ico ../logo.png

bin  etc  games  include  lib  lib64  libexec  n
[root@192 assets]# cat ../logo.png
56666 88888 99999 vvvvvv erfhiehgfiowe guihiouhi

bin  etc  games  include  lib  lib64  libexec  n
[root@192 assets]# rsync -av /usr/local/nginx-1.8.0/html /usr/local/nginx-1.8.0/backup/
```

```
[root@192 assets]# rsync -av /usr/local/nginx-1.8.0/html /usr/local/nginx-1.8.0/backup/
sending incremental file list
html/
html/favicon.ico
html/index.html
html/logo.png
html/assets/index.css
html/assets/index.js

sent 1,124 bytes  received 120 bytes  2,488.00 bytes/sec
total size is 702  speedup is 0.56
```

2.远程同步操作:

需求: 对/usr/local/nginx-1.8.0/html目录进行操作, 将其同步到node2的家目录的backup的目录中

注意: 需要双方均安装rsync, 同时配置了SSH密钥 (否则需要输入密码)

这个案例也是没有什么太大的问题的, 下面是我在这个案例中做出的

```
[root@192 backup]# rsync -avz /usr/local/nginx-1.8.0/html/ 192.168.149.131:/root/backup
sending incremental file list
./
favicon.ico
favion.ico
index.html
logo.png
assets/
assets/index.css
assets/index.js

sent 691 bytes  received 145 bytes  557.33 bytes/sec
total size is 702  speedup is 0.84
[root@192 backup]#
```

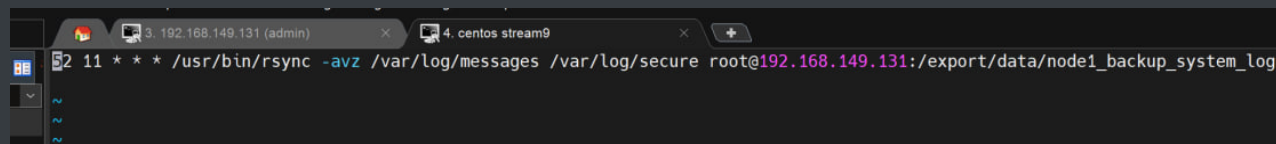
3.综合案例,实现定时与增量备份

需求说明:

在centos stream 9的 /var/log目录下, 存储了大量的关于系统的日志: 如messages文件存储大量的系统日志信息, secure日志文件存储了大量的与系统安全的日志。请对此目录中的messages、secure日志文件进行定时的备份到node2节点的/export/data/node1_backup_system_log目录中, 要求每日凌晨2点进行备份一次。

在这案例之中我们可以使用定时任务进行操作的

```
[root@192 backup]# crontab -e
```



The screenshot shows a terminal window with two tabs: '3. 192.168.149.131 (admin)' and '4. centos stream9'. The terminal displays a crontab entry: '2 11 * * * /usr/bin/rsync -avz /var/log/messages /var/log/secure root@192.168.149.131:/export/data/node1_backup_system_log'. The entry is being edited in a crontab file.

四.结合INOTIFY实现数据实时同步

首先将二个路径分别创建出来， 便于后续演示

node1执行: `mkdir -p /export/data/logs`

node2执行: `mkdir -p /export/data/node1_exe_backup_log`

```
failed to lock path /var/run/rsyncd.pid: resource temporarily unavailable
[root@192 admin]# mkdir -p /export/data/node1_exe_backup_log
[root@192 admin]#
```

```
[root@192 admin]# mkdir -p /export/data/logs
[root@192 admin]#
```

■ 优势:

高效性: `rsyncd` 的守护进程模式可以避免每次连接都启动 `SSH` 进程，因此在某些情况下可能会比 `SSH` 方式略微提高性能。

简化配置: 一旦配置好 `rsyncd`，无需每次都输入 `SSH` 密码，适合用于定时备份等自动化任务。

■ 缺点:

安全性问题: `rsyncd` 是基于开放端口的方式进行文件同步，默认情况下并不加密传输的数据。这使得数据传输过程可能暴露给中间人攻击（MITM）。虽然可以配置 `rsyncd` 使用 `--password-file` 来限制访问，但仍然不如 `SSH` 安全。

需要开放端口: 您需要在防火墙上开放 `rsync` 服务使用的端口（通常是 `873`），这可能增加潜在的安全风险。

配置复杂: 需要在服务器上配置 `rsyncd.conf`，并管理模块、权限、用户等细节。

1.对node2进行配置

1.创建用户

在这里面呢，我们需要在node2中创建2个用户，一个是`backup_user`，一个是`rsync`用户

```
useradd backup_user
useradd rsync
passwd backup_user
passwd rsync
```

注意密码，都设置为123456

2.修改配置文件

注意，这个配置文件要是在node2中修改的，以下是修改命令

```
# 全局设置
uid = rsync
gid = rsync
use chroot = no
port = 873
max connections = 200
pid file = /var/run/rsyncd.pid
log file = /var/log/rsyncd.log
transfer logging = yes
ignore nonreadable = yes
[node1_logs]
path = /export/data/node1_exe_backup_log
comment = Backup Directory
read only = no
secrets file = /etc/rsyncd.passwd
hosts allow = 192.168.149.0/24
hosts deny = *
auth users = backup_user
```

3.创建密码文件

```
echo "backup_user:123456" > /etc/rsyncd.passwd
# 设置密码文件的权限为600，确保该文件只有 RSYNC 服务可以读取
chmod 600 /etc/rsyncd.passwd
```

4. 启动rsync服务

```
1 rsync --daemon
```

```
[root@node1 ~]# chmod 600 /etc/rsyncd.passwd
[root@node1 ~]# rsync --daemon
[root@node1 ~]# ps -ef | grep rsync
root      379875      1   0 15:07 ?        00:00:00 rsync --daemon
root      380027    341670   0 15:08 pts/1    00:00:00 grep --color=auto rsync
[root@node1 ~]#
```

测试是否可以正常访问： 在node1执行

```
rsync rsync://backup_user@192.168.88.102/node1_logs
```

注意在node2需要开放端口：

```
firewall-cmd --permanent --add-port=873/tcp
```

```
[root@192 admin]# rsync rsync://backup_user@192.168.149.131/node1_logs
Password:
drwxr-xr-x          6 2026/07/24 19:36:57 .
[root@192 admin]#
```

五.设置开机自启动

1. 杀死进程

-- 查看进程

```
ps -ef | grep rsync -- 查看进程ID
```

-- 杀死进程

```
kill -9 rsync的进程ID
```

```
[root@192 admin]# ps -ef | grep rsync
root      22693      1   0 19:30 ?        00:00:00 rsync --daemon
root      32224    27796   0 20:42 pts/0    00:00:00 grep --color=auto rsync
[root@192 admin]# kill -22693 rsync
bash: kill: 22693: 无效的信号声明
[root@192 admin]# kill -9 22693
[root@192 admin]# vi /etc/systemd/system/rsyncd.service
[root@192 admin]# systemctl daemon-reload
[root@192 admin]# systemctl enable rsyncd
[root@192 admin]# systemctl start rsyncd
[root@192 admin]# systemctl status rsyncd
```


2.设置开机自启动

由于rsync并不是一个系统服务（简单判断：通过systemctl启动和关闭的服务），故需要先设置为系统服务后才可以设置

- 修改配置文件

```
[Unit]
Description=Rsync Daemon
After=network.target

[Service]
ExecStart=/usr/bin/rsync --daemon --config=/etc/rsyncd.conf --no-detach
ExecReload=/bin/kill -HUP $MAINPID
Restart=always

[Install]
WantedBy=multi-user.target
```

特殊说明：

ExecReload=/bin/kill -HUP \$MAINPID：通过发送 SIGHUP 信号，告诉 rsync 守护进程重新加载其配置文件，而无需停止并重新启动进程。

- 这个时候，我们需要把旧的日志文件给删掉

因为我们可能在操作的时候会产出一些错误的日志，就是由于这些错误的文件会导致，我们后续无法成功启动rsync服务

```
[root@192 admin]# systemctl status rsyncd
x rsyncd.service - Rsync Daemon
   Loaded: loaded (/etc/systemd/system/rsyncd.service; enabled; preset: disabled)
   Active: failed (Result: exit-code) since Fri 2026-07-24 20:46:12 CST; 9s ago
     Duration: 24ms
    Process: 33586 ExecStart=/usr/bin/rsync --daemon --config=/etc/rsyncd.conf --no-detach (code=exited, status=11)
   Main PID: 33586 (code=exited, status=11)
      CPU: 10ms
```

就像我这样的错误一样，这个时候，我们就要学会查看日志文件了，因为一些错误会记录在日志里面

```
[root@192 admin]# journalctl -xeu rsyncd.service
7月 24 20:49:56 192.168.149.131 rsyncd[34955]: rsync error: error in file IO (code 11) at clientserver.c(1469) [Receiver=3.2
7月 24 20:49:56 192.168.149.131 systemd[1]: rsyncd.service: Main process exited, code=exited, status=11/n/a
Subject: Unit process exited
Defined-By: systemd
Support: https://access.redhat.com/support

An ExecStart= process belonging to unit rsyncd.service has exited.

The process' exit code is 'exited' and its exit status is 11.
7月 24 20:49:56 192.168.149.131 systemd[1]: rsyncd.service: Failed with result 'exit-code'.
Subject: Unit failed
Defined-By: systemd
Support: https://access.redhat.com/support

The unit rsyncd.service has entered the 'failed' state with result 'exit-code'.
7月 24 20:49:57 192.168.149.131 systemd[1]: rsyncd.service: Scheduled restart job, restart counter is at 5.
Subject: Automatic restarting of a unit has been scheduled
Defined-By: systemd
Support: https://access.redhat.com/support

Automatic restarting of the unit rsyncd.service has been scheduled, as the result for
the configured Restart= setting for the unit.
7月 24 20:49:57 192.168.149.131 systemd[1]: Stopped Rsync Daemon.
Subject: rsyncd.service 单元已结束停止操作
Defined-By: systemd
Support: https://access.redhat.com/support

rsyncd.service 单元已结束停止操作。
7月 24 20:49:57 192.168.149.131 systemd[1]: rsyncd.service: Start request repeated too quickly.
7月 24 20:49:57 192.168.149.131 systemd[1]: rsyncd.service: Failed with result 'exit-code'.
Subject: Unit failed
Defined-By: systemd
Support: https://access.redhat.com/support

The unit rsyncd.service has entered the 'failed' state with result 'exit-code'.
7月 24 20:49:57 192.168.149.131 systemd[1]: Failed to start Rsync Daemon.
Subject: rsyncd.service 单元已失败
Defined-By: systemd
Support: https://access.redhat.com/support

rsyncd.service 单元已失败。
```

其实，当我把旧的日志文件给删掉之后哦，成功了，应该是之前启动失败过一次，日志文件，没有删除，然后再次启动，通过对比这个日志文件，发现问题，就无法启动，删除文件意思是把里面的的错误清理掉，之后相当于重新启动，所以可以成功启动后

```
● 结果为"Failed"。
[root@192 admin]# rm -f /var/run/rsyncd.pid
[root@192 admin]# systemctl start rsyncd
[root@192 admin]# systemctl status rsyncd
● rsyncd.service - Rsync Daemon
   Loaded: loaded (/etc/systemd/system/rsyncd.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-07-24 20:54:38 CST; 12s ago
     Main PID: 36609 (rsync)
       Tasks: 1 (limit: 16724)
      Memory: 872.0K (peak: 1.0M)
         CPU: 9ms
        CGroup: /system.slice/rsyncd.service
                └─36609 /usr/bin/rsync --daemon --config=/etc/rsyncd.conf --no-detach

7月 24 20:54:38 192.168.149.131 systemd[1]: Started Rsync Daemon.
7月 24 20:54:38 192.168.149.131 rsyncd[36609]: params.c:Parameter() - Ignoring badly formed line in config file: :# /etc/rsyncd: conf
7月 24 20:54:38 192.168.149.131 rsyncd[36609]: rsyncd version 3.2.5 starting, listening on port 873
...skipping...
● rsyncd.service - Rsync Daemon
   Loaded: loaded (/etc/systemd/system/rsyncd.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-07-24 20:54:38 CST; 12s ago
     Main PID: 36609 (rsync)
       Tasks: 1 (limit: 16724)
```

总结这次成功的逻辑

这次能成功，其实是两个动作配合的结果：

清理现场：`rm -f /var/run/rsyncd.pid` 清除了旧的工作证，扫清了路障。

修正配置：你之前（或者在我们的排查中）修正了 `/etc/rsyncd.conf` 里的错误（比如把密码文件的路径写对了，或者模块路径建好了）。

所以，并不是“这样修改就可以”这么简单，而是“清理了旧障碍 + 配置本身正确”共同作用的结果。

💡 小贴士：以后遇到类似问题怎么办？

下次如果再遇到服务启动失败，且提示 `already running`、`permission denied` 或 `status=11`，可以尝试这个“万能重启三部曲”：

查日志：`journalctl -xeu 服务名` 看具体报错。

删 pid 文件：`rm -f /var/run/服务名.pid` 清理旧状态。

重启服务：`systemctl restart 服务名`。

记住，日志是你最好的朋友，它能告诉你服务到底是哪里不舒服。现在你的 `rsync` 服务已经稳稳运行了，可以去客户端测试同步啦！

六.基于RSYNC服务完成手动同步

1.格式

用户名@主机地址::模块名称 或 `rsync://用户名@主机地址/模块名称`

说明：

模块名： 指的是配置到`rsync.conf` 中 **【】** 的内容

用户名： 指的是模块中指定的用户名， 如果没有指定， 可以是主机存在的用户名即可（前提是该用户需要有权限）

例如：

`backup_user@192.168.88.102::node1_logs`

2. 基于手动同步方式完成数据同步

```
rsync -avz /export/data/logs/ backup_user@node2::node1_logs
```

3. 创建日志文件

在创建的日志文件里面录入数据

```
cd /export/data/logs  
touch log.txt && echo "2491 797 1243124 9127">>log.txt
```

4. 开始传输文件了

在这一步里面呢，我们需要注意上下文，把这个权限给关掉，否则，后面传输就像我一样不会成功的

```
[root@192 logs]# rsync -avz /export/data/logs/ rsync://backup_user@192.168.149.131/node1_logs  
Password:  
sending incremental file list  
rsync: [generator] failed to set times on "." (in node1_logs): Permission denied (13)  
./  
log.txt  
rsync: [receiver] mkstemp ".log.txt.mRLP0Y" (in node1_logs) failed: Permission denied (13)  
  
sent 224 bytes  received 223 bytes  127.71 bytes/sec  
total size is 89  speedup is 0.20  
rsync error: some files/attrs were not transferred (see previous errors) (code 23) at main.c(1327) [sender=3.2.5]  
[root@192 logs]#
```

大家可以看到，我通过传送指令，无法发送成功，但是文件的权限都是开的，是最大的权限

```
[root@192 logs]# ssh backup_user@192.168.149.131~  
ssh: Could not resolve hostname 192.168.149.131~: Name or service not known  
[root@192 logs]# ssh backup_user@192.168.149.131  
backup_user@192.168.149.131's password:  
[backup_user@192 ~]$ ls -ld /export/data/node1_exe_backup_log/  
drwxrwxrwx. 2 rsync rsync 6  7月 25 08:46 /export/data/node1_exe_backup_log/  
[backup_user@192 ~]$
```

当时，我就比较迷了，不知道该如何办，然后我问了千问，它给予了我很大的帮助了，我明白了其实是我Linux的上下文的权限比较严格，需要将其关闭

```
[backup_user@192 ~]$ sudo getenforce
Enforcing
[backup_user@192 ~]$
```

解决方案

以下是干问告诉我的

找到根本原因了！

SELinux 处于 `Enforcing`（强制）模式**，这就是 `rsync` 报 `Permission denied` 的真正原因。

即使目录权限是 `drwxrwxrwx`（777），SELinux 的安全策略仍然会阻止 `rsync` 进程写入文件。

解决方案

方案1：临时关闭 SELinux（快速验证）

在远程服务器 `192.168.149.131` 上执行：

```
# 临时设置为 Permissive 模式（不需要重启）
sudo setenforce 0

# 验证
getenforce

# 应该输出：Permissive
```

然后重新执行 `rsync` 命令，应该就能成功了。

⚠ 这只是临时生效，重启后会恢复为 `Enforcing`。

方案2：永久关闭 SELinux

```
# 编辑 SELinux 配置文件
sudo vi /etc/selinux/config

# 将 SELINUX=enforcing 改为:
SELINUX=disabled

# 重启服务器生效
sudo reboot
```

方案3：设置正确的 SELinux 上下文（推荐，更安全）

如果不想关闭 SELinux，可以给目录设置正确的标签：

```
# 查看当前目录的 SELinux 上下文
ls -Zd /export/data/node1_exe_backup_log/

# 设置 rsync 可以写入的上下文
sudo chcon -Rt public_content_rw_t /export/data/node1_exe_backup_log/

# 如果 rsync 以守护进程方式运行，还需要：
sudo setsebool -P rsync_full_access on
```

建议

先用方案1快速验证，确认是 SELinux 的问题后，再决定用方案2还是方案3长期解决。

```
# 在远程服务器上执行  
sudo setenforce 0
```

然后回到本地重新执行 rsync 命令即可。

```
[rsync@192 data]$ sudo setenforce 0
```

我们信任您已经从系统管理员那里了解了日常注意事项。
总结起来无外乎这三点：

- #1) 尊重别人的隐私。
- #2) 输入前要先考虑(后果和风险)。
- #3) 权力越大，责任越大。

为安全起见，您键入的密码将不可见。

```
[sudo] rsync 的密码：  
sudo: 需要密码
```

这个是我按照，他的提示放权的步骤

接下来，很明显，我再次发送文件，直接成功了

```
Connection to 192.168.1.131 closed.  
[root@192 logs]# rsync -avz /export/data/logs/ rsync://backup_user@192.168.149.131/node1_logs  
Password:  
sending incremental file list  
./  
log.txt  
  
sent 223 bytes  received 38 bytes  74.57 bytes/sec  
total size is 89  speedup is 0.34  
[root@192 logs]#
```

```
[root@192 data]# ls  
node1_backup_system_log node1_exe_backup_log  
[root@192 data]# cd node1_exe_backup_log/  
[root@192 node1_exe_backup_log]# ls  
log.txt  
[root@192 node1_exe_backup_log]# cat log.txt  
128 6 2026/07/24 19:36:57 .  
rsync -avz /export/data/logs/ backup_user@node2::node1_logs  
[root@192 node1_exe_backup_log]#
```

注意，ai说的是下一次重启还会权限不足，因此我们需要修改配置文件

```
[root@192 data]# vi /etc/selinux/config  
[root@192 data]#
```

```
[root@192 node1_exe_backup_log]# cat /etc/selinux/config

# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#   enforcing - SELinux security policy is enforced.
#   permissive - SELinux prints warnings instead of enforcing.
#   disabled - No SELinux policy is loaded.
# See also:
# https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/9/html/using_selinux/changing-selinux-modes-at-boot-time_changing-selinux-states-and-modes
#
# NOTE: Up to RHEL 8 release included, SELINUX=disabled would also
# fully disable SELinux during boot. If you need a system with SELinux
# fully disabled instead of SELinux running with no policy loaded, you
# need to pass selinux=0 to the kernel command line. You can use grubby
# to persistently set the bootloader to boot with selinux=0:
#
#   grubby --update-kernel ALL --args selinux=0
#
# To revert back to SELinux enabled:
#
#   grubby --update-kernel ALL --remove-args selinux
#
SELINUX=disabled
# SELINUXTYPE= can take one of these three values:
#   targeted - Targeted processes are protected,
#   mls - Multi Level Security protection.
SELINUXTYPE=targeted

[root@192 node1_exe_backup_log]# █
```

将SELINUX修改为disabled就可以了

然后，我们重启一起机器，是配置生效，就可以了。

七,使用密码文件， 实现不需要输入密码

■ 步骤一： 配置密码文件

```
echo '123456' > /etc/rsync.password
```

```
chmod 600 /etc/rsync.password
```

注意： 权限必须是600，否则会报错

```
[root@192 logs]# echo "1239 79723 70923 gcgiix 68xyxep}exr ">>b.txt
[root@192 logs]# ll
```

```
[root@192 etc]# echo "123456">>rsync.password
[root@192 etc]# chmod 600 rsync.password
rsyncd.conf rsyncd.passwd rsync.password rsyslog.conf rsyslog.d/
[root@192 etc]# chmod 600 rsync.password
```

■ 步骤二： 使用密码文件， 实现不需要输入密码


```
rsync -avz --password-file=/etc/rsync.password /export/data/logs/  
backup_user@node2::node1_logs
```

```
[root@192 etc]# chmod 600 /etc/rsync.password  
[root@192 etc]# rsync -avz --password-file=/etc/rsync.password /export/data/logs/ rsync://backup_user@192.168.149.131/node1_logs  
sending incremental file list  
./  
b.txt  
  
sent 1,143 bytes  received 38 bytes  2,362.00 bytes/sec  
total size is 2,634  speedup is 2.23  
[root@192 etc]#
```

查看node2机器

```
[root@192 node1_exe_backup_log]# ll  
总用量 8  
-rw-r--r--. 1 rsync rsync 2545  7月 25 10:04 b.txt  
-rwxrwxrwx. 1 rsync rsync   89  7月 24 21:39 log.txt  
[root@192 node1_exe_backup_log]#
```

八.使用主机名

这个应该修改配置文件hosts

```
echo "192.168.149.128 node1 node1.itcast.cn">>/etc/hosts  
echo "192.168.149.131 node2 node2.itcast.cn">>/etc/hosts
```

完毕！

```
[root@192 etc]# vim /etc/hosts
```

```
[root@192 etc]# ping node2  
PING node2 (192.168.149.131) 56(84) 比特的数据。  
64 比特，来自 node2 (192.168.149.131): icmp_seq=1 ttl=64 时间=0.870 毫秒  
64 比特，来自 node2 (192.168.149.131): icmp_seq=2 ttl=64 时间=0.711 毫秒  
64 比特，来自 node2 (192.168.149.131): icmp_seq=3 ttl=64 时间=0.527 毫秒  
64 比特，来自 node2 (192.168.149.131): icmp_seq=4 ttl=64 时间=0.880 毫秒  
^C
```

```
total size is 2,000, speedup is 20000
[root@192 etc]# cat /etc/hosts
127.0.0.1    localhost localhost.localdomain localhost4 localhost4.localdomain4
::1        localhost localhost.localdomain localhost6 localhost6.localdomain6
192.168.149.128 node1 node1.itcast.cn
192.168.149.131 node2 node2.itcast.cn

[root@192 etc]# █
```

九.使用脚本自动配置

1.安装使用的软件

- 安装第三方源

首先启用EPEL仓库：

```
dnf install epel-release
```

- 安装监视软件

安装inotify-tools

```
dnf install -y inotify-tools
```

2.编写脚本

- 1.vi /root/real_time_sync.sh

创建sh脚本文件

```
#!/bin/bash

# 定义源目录和目标目录（目标目录是远程服务器的目录）
SOURCE_DIR="/export/data/logs"

TARGET_DIR="root@192.168.88.102:/export/data/node1_exe_backup_log" # 替换为目标服务器的用户名和目录

# 日志文件
LOG_FILE="/var/log/rsync_realtime_sync.log"

# 使用inotifywait监听${WATCH_DIR}目录中的文件变化
inotifywait -m -r -e modify,create,delete,move "${SOURCE_DIR}" | while read
```

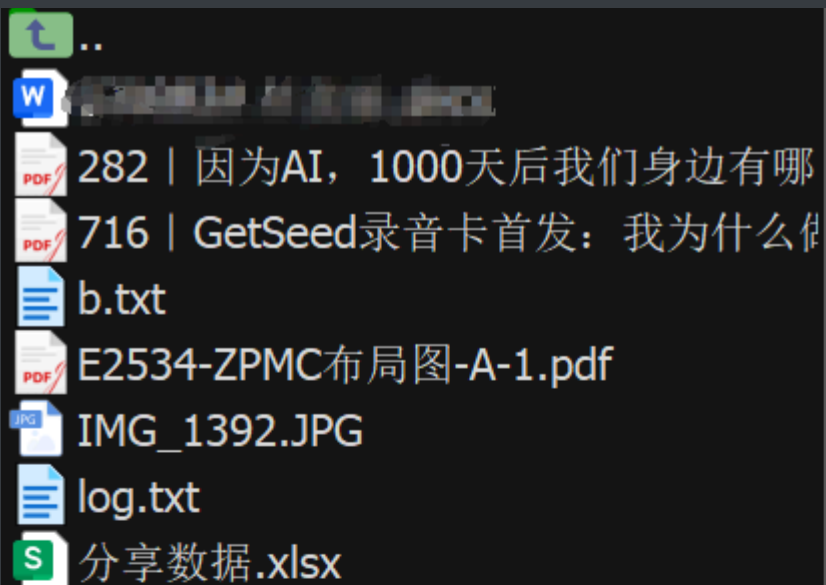
```
path action file; do
echo "Detected ${action} on ${file} in ${path}. Starting rsync..." >>
"$LOG_FILE"
# 执行rsync同步操作
rsync -avz --port=873 --delete -e ssh "${SOURCE_DIR}/" "${TARGET_DIR}"
>> "$LOG_FILE" 2>&1
echo "Sync completed for ${file} in ${path}." >> "$LOG_FILE"
done
```

3.给文件加权以及启动脚本文件

```
完毕！
[root@192 etc]# vi /root/real_time_sync.sh
[root@192 etc]# chmod +x /root/real_time_sync.sh
[root@192 etc]# ./real_time_sync.sh
bash: ./real_time_sync.sh: 没有那个文件或目录
[root@192 etc]# ./root/real_time_sync.sh
bash: ./root/real_time_sync.sh: 没有那个文件或目录
[root@192 etc]# cd ~
[root@192 ~]# ./real_time_sync.sh
Setting up watches. Beware: since -r was given, this may take a while!
Watches established.
```

4.最后测试一下

在node1的/export/data/logs 里面上传几个文件，然后再打开一个node1服务器开启tail -100f监听文件，查看是否有修改，以及node2是否也跟着变化呢



```
Sync completed for 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/.
Detected MODIFY on 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/. Starting rsync...
sending incremental file list

sent 450 bytes received 12 bytes 924.00 bytes/sec
total size is 15,397,560 speedup is 33,328.05
Sync completed for 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/.
Detected MODIFY on 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/. Starting rsync...
sending incremental file list

sent 450 bytes received 12 bytes 308.00 bytes/sec
total size is 15,397,560 speedup is 33,328.05
Sync completed for 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/.
Detected MODIFY on 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/. Starting rsync...
sending incremental file list

sent 450 bytes received 12 bytes 924.00 bytes/sec
total size is 15,397,560 speedup is 33,328.05
Sync completed for 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/.
Detected MODIFY on 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/. Starting rsync...
sending incremental file list

sent 450 bytes received 12 bytes 924.00 bytes/sec
total size is 15,397,560 speedup is 33,328.05
Sync completed for 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/.
Detected MODIFY on 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/. Starting rsync...
sending incremental file list

sent 450 bytes received 12 bytes 924.00 bytes/sec
total size is 15,397,560 speedup is 33,328.05
Sync completed for 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/.
Detected MODIFY on 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/. Starting rsync...
sending incremental file list

sent 450 bytes received 12 bytes 308.00 bytes/sec
total size is 15,397,560 speedup is 33,328.05
Sync completed for 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/.
Detected MODIFY on 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/. Starting rsync...
sending incremental file list

sent 450 bytes received 12 bytes 924.00 bytes/sec
total size is 15,397,560 speedup is 33,328.05
Sync completed for 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf in /export/data/logs/.

```

```
[root@192 node1_exe_backup_log]# ll
总用量 15052
-rw-r--r--. 1 admin admin 6428892 7月 25 11:00 282 | 因为AI，1000天后我们身边有哪些确定会发生的变化（上）.pdf
-rw-r--r--. 1 admin admin 843124 7月 25 11:09 716 | GetSeed录音卡首发：我为什么做AI硬件？.pdf
-rw-r--r--. 1 admin admin 2321833 7月 25 11:09 分享数据.xlsx
-rw-r--r--. 1 admin admin 3514 7月 25 11:00 b.txt
-rw-r--r--. 1 root root 2545 7月 25 10:04 E2534-ZPMC布局图-A-1.pdf
-rw-r--r--. 1 admin admin 1586168 7月 25 11:09 IMG_1392.JPG
-rw-r--r--. 1 admin admin 4211395 7月 25 11:01 log.txt
-rwxrwxrwx. 1 rsync rsync 89 7月 24 21:39
[root@192 node1_exe_backup_log]#
```

最后，发现成功在node2里面显示，这个实验就完成了